Exercício 1 — Atribuir variáveis aos elementos da sequência de insulina humana

Objetivo

Neste exercício, você irá praticar:

- Criação e atribuição de variáveis;
- Strings;
- Concatenação de strings;
- Convenções de nomenclatura no Python;
- Organização do código seguindo boas práticas.

Passo a passo

1. Abra seu arquivo Python

No **Explorer** do VS Code, abra o arquivo `.py` criado anteriormente.

No início do arquivo, adicione comentários informando:

- A versão do Python utilizada;
- A codificação do arquivo.

> **Dica:** comentários em Python começam com `#`.

2. Crie a variável `preproInsulin`

Adicione um comentário explicando que a próxima variável armazenará a sequência da pré-proinsulina humana.

Depois, crie uma variável chamada:

`preproInsulin`

Atribua a ela a sequência fornecida no exercício.

A sequência deverá ser armazenada como uma **string**.

> **Dica:** strings devem estar entre aspas.

Como a sequência é longa, ela foi dividida em duas partes no exercício original.

Observe o uso da barra invertida (`\`) para continuar a expressão na linha seguinte.

> **Por que isso é útil?**
> O PEP 8 recomenda manter as linhas de código com, no máximo, 79 caracteres, favorecendo a legibilidade.

3. Crie as variáveis das partes da insulina

Adicione um comentário indicando que você irá armazenar as partes restantes da sequência da insulina.

Crie as seguintes variáveis:

Variável

O que armazenar

`lsInsulin`

Primeira parte da sequência

`bInsulin`

Segunda parte da sequência

`aInsulin`

Terceira parte da sequência

```
cInsulin
```

Quarta parte da sequência

Utilize as sequências fornecidas no enunciado.

> **Atenção:** respeite exatamente as letras maiúsculas e minúsculas dos nomes das variáveis.

4. Observe a nomenclatura

Os nomes utilizados seguem um padrão de escrita conhecido como **camelCase**.

Por exemplo:

```
minhaVariavel
```

Nesse padrão, a primeira palavra começa com letra minúscula e as palavras seguintes começam com letra maiúscula.

> **Dica:** consistência na nomenclatura facilita a leitura e manutenção do código.

5. Combine partes da sequência

Agora crie uma variável chamada:

```
insulin
```

Ela deverá armazenar a combinação de duas das sequências menores.

Para juntar strings, utilize o operador:

```
+
```

> **Dica:** nesse caso, o `+` não está realizando uma soma numérica. Ele está **concatenando strings**.

Exercício 3 — Usar `print()` para exibir as sequências

Objetivo

Neste exercício, você irá praticar:

- Uso da função `print()`;
- Exibição de strings;
- Exibição de variáveis;
- Concatenação de strings.

Passo a passo

1. Adicione um comentário

No seu arquivo Python, adicione um comentário explicando que você irá imprimir a sequência da insulina humana no console.

2. Exiba um texto no console

Utilize `print()` para exibir uma mensagem informando que será apresentada a sequência da pré-proinsulina humana.

> **Dica:** nesse primeiro `print()`, você estará passando diretamente uma string para a função.

3. Exiba uma variável

Na linha seguinte, utilize `print()` novamente, mas dessa vez passe a variável que contém a sequência da pré-proinsulina.

Observe a diferença entre:

```
print("algum texto")
```

e:

```
print(umaVariavel)
```

No primeiro caso, o Python imprime o texto informado diretamente.

No segundo, ele imprime o conteúdo armazenado na variável.

4. Concatene texto e variável

Agora exiba uma mensagem informando qual cadeia da insulina será apresentada e, em seguida, o conteúdo armazenado em `aInsulin`.

Utilize o operador `+` para concatenar as duas strings.

A estrutura deve seguir esta ideia:

```
print("texto" + variável)
```

5. Observe outra possibilidade

O `print()` também permite receber vários argumentos separados por vírgula.

Por exemplo:

```
print("texto:", variável)
```

Compare essa abordagem com a concatenação utilizando `+`.

6. Execute o programa

Salve o arquivo e execute o código.

Confira se:

- A sequência da pré-proinsulina aparece;
- A sequência da cadeia de insulina aparece;
- Os textos estão sendo exibidos corretamente;
- Não existem erros de sintaxe.

Exercício 4 — Calcular o peso molecular aproximado da insulina

Objetivo

Neste exercício, você irá utilizar um código fornecido para calcular uma estimativa do peso molecular da insulina.

Além de executar o código, observe conceitos que serão utilizados posteriormente, como:

- Dicionários;
- Listas;
- `count()`;
- `sum()`;
- Compreensão de dicionários;
- Conversão de tipos com `str()`.

Passo a passo

1. Adicione o código fornecido

Com o arquivo `.py` aberto, vá até o final do código e adicione o trecho fornecido no exercício.

Não é necessário criar o algoritmo do zero neste momento.

O objetivo principal é **entender o que cada parte do código está fazendo**.

2. Observe o dicionário de pesos

A primeira parte cria uma estrutura chamada:

```
aaWeights
```

Ela associa cada letra de aminoácido ao seu respectivo peso.

Por exemplo, conceitualmente:

```
'A' → peso do aminoácido A
'C' → peso do aminoácido C
```

> **Dica:** essa estrutura utiliza um **dicionário**, no qual existe uma relação entre uma chave e um valor.

3. Observe a contagem dos aminoácidos

A variável:

```
aaCountInsulin
```

é utilizada para descobrir quantas vezes cada aminoácido aparece na sequência de insulina.

O código utiliza:

```
count()
```

para realizar essa contagem.

> **Pergunta:** se a letra `A` aparecer 10 vezes na sequência, qual deveria ser o valor armazenado para `A` ?

4. Observe o cálculo do peso molecular

Depois de descobrir quantas vezes cada aminoácido aparece, o programa utiliza os pesos armazenados anteriormente.

A lógica geral é:

```
quantidade do aminoácido × peso do aminoácido
```

Depois, os resultados são somados para chegar ao peso molecular aproximado.

> **Não é necessário memorizar todo o código neste momento.** Tente identificar apenas o papel de cada variável.

5. Execute o programa

Salve e execute o arquivo.

Observe o peso molecular apresentado no console.

O valor calculado pelo programa será aproximadamente:

```
6696.42
```

Esse valor é diferente do peso molecular real informado no exercício:

```
5807.63
```

Isso acontece porque o cálculo utilizado pelo programa é uma **aproximação** e não considera determinados processos e ligações presentes na molécula real.

---

6. Calcule o percentual de erro

Agora crie uma variável para armazenar o peso molecular real:

```
5807.63
```

Em seguida, utilize a fórmula fornecida no exercício:

```
percentual de erro = (|medido - aceito| / aceito) × 100
```

No código, você deverá utilizar o valor calculado anteriormente e o valor real para realizar essa operação.

> **Dica:** o valor calculado anteriormente está armazenado na variável `molecularWeightInsulin`.

7. Exiba o resultado

Utilize `print()` para mostrar o percentual de erro.

Como o resultado do cálculo será um número decimal e você deseja concatená-lo com um texto, será necessário convertê-lo para `str`.

A estrutura geral será semelhante a:

```
print("texto" + str(resultado))
```

> **Por que utilizar `str()`?**
> O operador `+` utilizado para concatenar strings espera que os valores envolvidos sejam strings. A função `str()` converte o resultado numérico para texto.

8. Execute novamente

Salve e execute o programa.

Verifique se o percentual de erro é apresentado corretamente.

Checklist

Exercício 1

- Criei a variável `preproInsulin`.
- Armazenei a sequência como uma string.
- Criei as variáveis das diferentes partes da insulina.
- Utilizei os nomes das variáveis corretamente.
- Combinei strings utilizando `+`.

Exercício 3

- Utilizei `print()` para exibir textos.
- Utilizei `print()` para exibir uma variável.
- Concatenei uma string com uma variável.
- Executei o programa e analisei a saída.

Exercício 4

- Adicionei e executei o código fornecido.
- Identifiquei o dicionário de pesos dos aminoácidos.
- Entendi que `count()` é utilizado para contar aminoácidos.

- Identifiquei como o programa calcula o peso molecular.
- Armazenei o peso molecular real em uma variável.
- Calculei o percentual de erro.
- Utilizei `str()` para converter o resultado numérico em string.
- Executei o programa e verifiquei os resultados.